Graphs, Ontology, and Reasoning with GestaltDB

Mylonas Charilaos

Oct 24, 2026

READING GUIDE

1 Graph databases, knowledge graphs, and meaning **4**
- 1.1 Why store a graph? . . . 4
- 1.2 Choosing nodes, edges, labels, and properties . . . 5
- 1.3 The technology landscape . . . 5
- 1.4 Stage 1 — the property-graph world . . . 5
- 1.5 Stage 2 — the semantic overlay . . . 5
- 1.6 Stage 3 — the reasoning layer . . . 8
- 1.7 Theory and further reading . . . 8
- 1.8 Check your understanding . . . 10

2 An ontology turns vocabulary into implications **11**
- 2.1 Why a schema is more than a set of labels . . . 11
- 2.2 Reading the notation . . . 11
- 2.3 From one assertion to two consequences . . . 12
- 2.4 Refinement needs evidence about the job . . . 13
- 2.5 How much of the domain have we actually defined? . . . 14
- 2.6 What compilation contributes . . . 16
- 2.7 Theory and further reading . . . 16
- 2.8 Check your understanding . . . 16

3 How a reasoning engine works **17**
- 3.1 Why not just write another Python loop? . . . 17
- 3.2 From a sentence to a rule . . . 17
- 3.3 Matching a rule is a join . . . 18
- 3.4 Why recursion needs a fixed point . . . 18
- 3.5 Try a join and a recursive rule . . . 20
- 3.6 What makes the answer trustworthy? . . . 20
- 3.7 Avoiding unnecessary work . . . 21
- 3.8 Why intermediate representations exist . . . 21
- 3.9 Other engines: similar notation, different jobs . . . 22
- 3.10 Check your understanding . . . 22
- 3.11 Theory behind the explanations . . . 23

4 Connecting a database to a reasoner **24**
- 4.1 Why the engine should not read an arbitrary live graph . . . 24
- 4.2 A complete stored-graph example . . . 24
- 4.3 When results should become durable . . . 25
- 4.4 Why extraction can dominate performance . . . 26
- 4.5 Check your understanding . . . 26

5 Claims, time, and belief **27**
5.1 Why a triple is sometimes not enough . . . 27
5.2 Missing evidence is not a negative assertion . . . 27
5.3 Combining time with derivations . . . 28
5.4 Why deleting an assertion can be difficult . . . 28
5.5 Belief and confidence answer different questions . . . 28
5.6 Theory and further reading . . . 29
5.7 Check your understanding . . . 29

6 Querying and sampling with inferred facts **30**
6.1 Which graph does an answer come from? . . . 30
6.2 Why sampling is a different operation . . . 30
6.3 Theory and further reading . . . 31
6.4 Check your understanding . . . 31

7 The cost of reasoning: what becomes expensive, and why **32**
7.1 Why a fast rule is not necessarily a fast application . . . 32
7.2 Joins: avoid combinations before making them faster . . . 32
7.3 Some costs are inherent in the requested answer . . . 33
7.4 Full recomputation versus maintenance . . . 33
7.5 Reading measurements without overgeneralizing . . . 33
7.6 Check your understanding . . . 34

8 Choosing a supported reasoning workflow **35**
8.1 Start with the question, not with a standards label . . . 35
8.2 Some problems need a different engine family . . . 35
8.3 An available feature may not compose with every other one . . . 36
8.4 Integration pain points to plan for . . . 36
8.5 What you should now be able to explain . . . 37
8.6 Check your understanding . . . 37

9 Glossary: the distinctions that matter **38**

10 Tutorial notebooks **40**

Looking for installation instructions or the API reference? Visit the GestaltDB library documentation. This tutorial is a separate, self-contained learning guide.

Imagine building a research directory from three sources. A personnel system lists appointments, a grant system identifies investigators, and a public catalog records which laboratories belong to which institutions. Each source has useful information, but none answers the product question by itself: "Which people are connected to this institution, and why do we believe that?"

At first, it is tempting to join a few tables and put the result behind an API. That may be exactly the right solution. The difficulty grows when the sources use different vocabularies, relationships span several hops, records arrive at different times, or an answer must include its evidence. One source calls Alice a researcher, another asks for people. Acme Lab belongs to a group that belongs to a university. A correction to one appointment should change the answers that depended on it, without erasing answers supported elsewhere.

These are several different problems, not one problem solved by choosing a database brand. A **graph model** makes identities and relationships explicit. A **graph database** persists that model and retrieves connected records. A **query language** states which records or paths an application wants. None of those choices alone tells the system that every researcher is a person. For that, we need explicit meaning: a vocabulary and semantic assumptions, often expressed by an **ontology**. A **reasoning engine** applies those assumptions systematically, so consequences need not be reimplemented in every report or endpoint. Together, such choices can support a **knowledge graph**—a representation of domain knowledge, not merely a particular storage format.

The distinction becomes practical immediately. Storing one more label on Alice makes one query work, but now the application must keep that label in sync when the schema or supporting records change. Repeating the same logic inside every query avoids that copy but spreads the policy across the system. Computing a reusable logical closure offers another trade-off: share the rules and answers, while paying for inference, context binding, and maintenance. This tutorial develops those choices instead of treating inference as a feature that should always be switched on.

GestaltDB provides a concrete setting in which to learn the distinctions. It has an embedded property graph, Cypher queries, an explicit ontology-to-rule compiler, standalone reasoning APIs, and separate adapters for stored data and result consumption. We will start with one fact about Alice, follow it through a complete inference, then ask what changes when data is large, historical, contradictory, or used for sampling. The goal is both to use GestaltDB and to recognize the same underlying design problems in other databases and reasoning engines.

The intended reader is comfortable with Python, dictionaries, SQL-style joins, and basic data structures. No previous RDF, OWL, or logic-programming experience is assumed. Equations and execution details are included for readers who want to understand the implementation costs, but the first pass can focus on the worked examples and their explanations.

What you will learn

By the end, you should be able to:

- distinguish a graph data model, a graph database, and a knowledge graph;
- choose what to represent as entities, relationships, labels, or properties;
- explain where RDF, OWL, Cypher, GraphQL, SQL, Datalog, and Prolog fit;
- read basic rule notation and trace joins and recursion to a fixed point;
- separate a stored fact, an inferred fact, a proof witness, and a source claim;
- decide when to compute a closure, query a subset, or build a sampling snapshot;
- recognize unsupported combinations and the costs that optimizations cannot remove.

The running example

Alice works for ACME. We record a `worksFor` relationship that explicitly means employment—not merely being present at ACME's premises. Its domain is `Employee`, and every `Employee` is a `Person`. Can the application infer Alice's membership without storing either label? What more would it need to classify her as a `Researcher`?

The second question is just as important as the first. Employment alone does not establish a research role. We will add a separate `hasJobDescription` fact, with the simplified categories `researcher` and `not_researcher`, and a rule that refines an employee into a researcher only with positive research-job evidence. Missing job information remains missing information, not a negative classification. These distinctions let us see both what an ontology implies and what it must **not** imply.

How to read this document

First pass: build the mental model. Read chapters 1–4 in order: graph and knowledge-graph concepts, ontology implications, the reasoning engine, then the complete database example. Run notebook 1 after chapter 2 if desired, and continue to chapter 6 to understand how applications consume the answer. You should be able to draw the example and explain each inferred membership without reading the library source.

Second pass: study the trade-offs. Revisit chapter 3's execution strategies and engine comparisons. Chapter 5 adds sources, updates, and time; chapter 7 explains performance. Chapter 8 helps choose a supported workflow rather than promising a universal reasoner. The glossary provides a quick reference, and each chapter ends with a self-check and an answer.

Running the examples

This is a standalone tutorial: its chapters introduce the concepts and supply the worked examples without requiring you to read the library manual first. For API details and installation alternatives, consult the GestaltDB library documentation and library repository. External library references follow the current library and

may describe APIs newer than the checkout tested here; they are optional further reading, not prerequisites.

The examples target the source checkout accompanying this tutorial, including its `gestalt-core-v1` ontology fragment. Do not assume that an older PyPI release has the same APIs. Use Python 3.10–3.13. From the repository root:

```
uv sync --extra lmdb
uv run --with jupyterlab --with ipykernel jupyter lab docs/onto-tutorial/
↪notebooks
```

The standalone reasoning examples need no embedded backend. Database examples use the explicitly installed LMDB backend and disposable temporary directories. Select a Jupyter kernel from this environment. You can also just read the code: assertions in the examples state the expected results.

CHAPTER

ONE

GRAPH DATABASES, KNOWLEDGE GRAPHS, AND MEANING

1.1 Why store a graph?

Suppose an application asks “Which people work at organizations connected to this research group?” The interesting part is the connections, not one record in isolation. A graph makes those connections explicit: entities are nodes, relationships are edges, and a path joins several relationships into a route. Direction matters: Alice working for Acme does not mean Acme works for Alice.

A graph database stores and queries such structures. That does not make it automatically a **knowledge graph**. The latter describes the *representation of domain knowledge*: which things identifiers denote, what relationship names mean, and how statements can be combined or interpreted. The term has several usages; a knowledge graph need not use RDF, a formal ontology, or any inference engine. A property graph can represent one, as can RDF stored in a triple store.

Keep these questions separate:

Table 1: Three layers, three responsibilities

Layer	Question	In our example
Graph model	What are the entities and connections?	Alice `worksFor` Acme.
Database	How are they stored and retrieved consistently?	Persist an edge; query it through Cypher.
Domain semantics	What follows from the chosen meaning?	Employment implies Employee membership, not a research role.

SQL tables can encode these same relationships using keys and joins. A graph database does not create information unavailable to a relational representation, nor is every graph query inherently faster. It changes the interface and storage choices around relationship-heavy workloads. A reasoning engine adds another service: consequences of explicit rules, independent of the storage format.

1.2 Choosing nodes, edges, labels, and properties

Give something a node when its identity and connections matter: Alice and Acme must be recognizable across many statements. Give `Alice` a display-name property rather than using the spelling as identity; two people can share that name. `Employee` can be a label for storage/query purposes. Treating it as an ontology class requires an explicit mapping, not a suggestive string.

The same choice affects facts about relationships. A physical `worksFor` edge can have an ID, a source, a start date, and a role. Collapsing it to a logical triple loses those distinctions unless they are modeled separately. If two employment records have the same endpoints, they are two records but may support the same proposition. This distinction will matter for provenance and deletion in later chapters.

1.3 The technology landscape

Intuition first. Picture a whiteboard: sticky notes for things (`alice`, `acme`), arrows between them labeled `worksFor`. That is a *property graph* — gestaltdb's home turf (`src/gestaltdb/graphdb.py`): `Node` objects carry native labels, `Edge` objects carry their relationship in the `properties["type"]` convention. Now lay *tracing paper* over the whiteboard and copy every arrow as a uniform triple `(subject, predicate, object)`; write class membership as just another arrow, `(alice, rdf:type, Employee)`. That tracing is the *semantic* view. In the following maps, colors and enclosing groups distinguish conceptual layers; small badges describe each node's role. Read each arrow as a labeled relationship, not as an unconditional data-conversion pipeline. The picture grows in three stages so it need not be learned all at once.

1.4 Stage 1 — the property-graph world

`Property Graph DB` *stores* labeled nodes and typed edges. **Cypher** expresses graph patterns (GestaltDB's query language; see the library's Cypher reference). **GraphQL** describes a typed API response: application resolvers may read a graph, a relational database, or remote services. It is not a native property- graph query language. **SQL** queries relations; **SQL/PGQ** adds property-graph pattern matching to SQL implementations that support it. GestaltDB supplies neither SQL/PGQ nor a GraphQL server. The map compares technologies, not a list of supported endpoints.

Intuition: a query language is a way to ask about the whiteboard; an API is a way to package answers. Naming an arrow `worksFor` does not by itself add domain, range, or subclass inference.

1.5 Stage 2 — the semantic overlay

Selected property nodes/edges *project to* binary facts with an explicit mapping. **RDF** represents graphs as sets of triples: subjects are IRIs or blank nodes, predicates are IRIs, and objects may also be literals. A bare `Fact("alice", "worksFor", "acme")` is a logical triple over local symbols, not automatically a

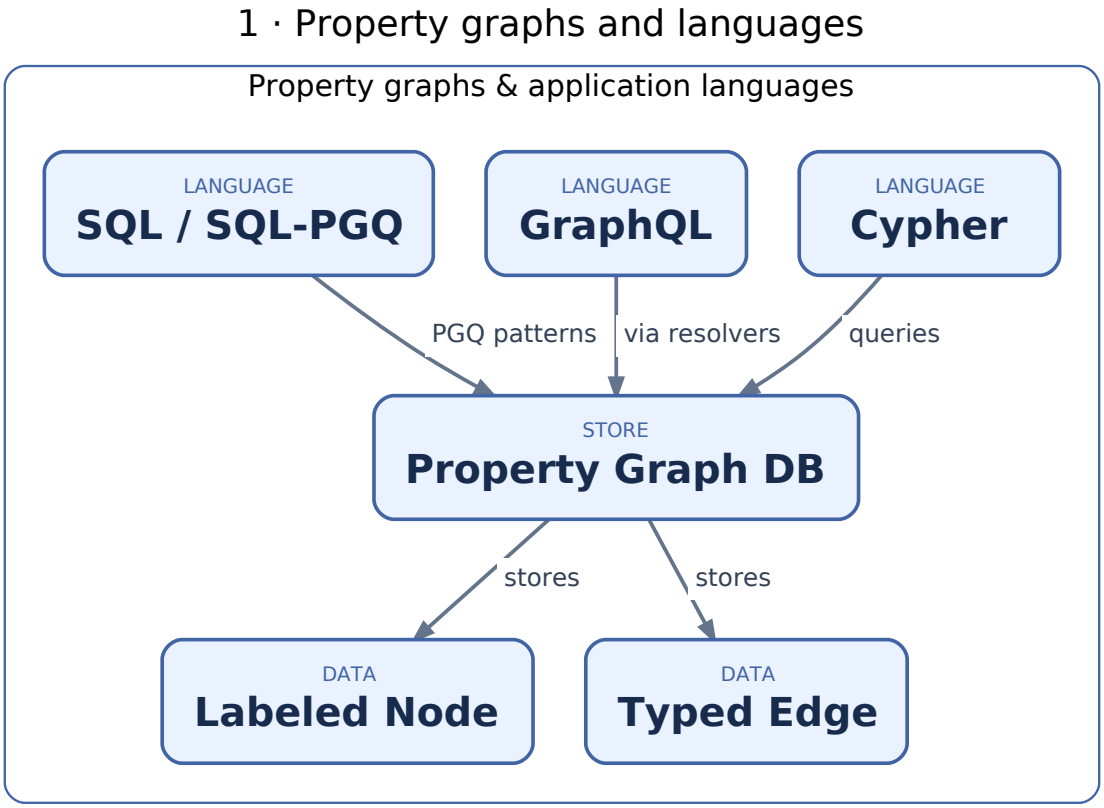


Fig. 1: Stage 1: graph storage and related language/API families.

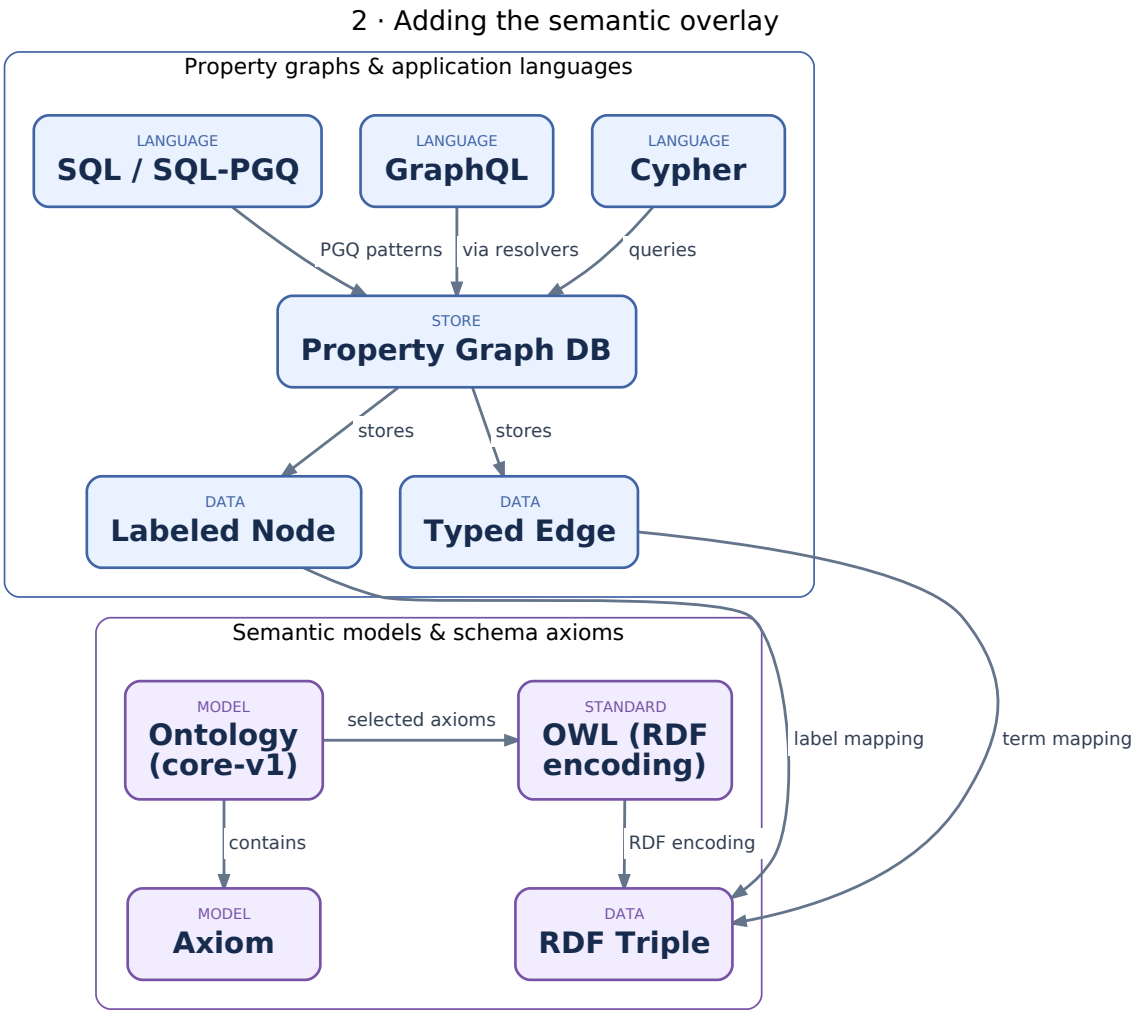


Fig. 2: Stage 2: projecting the whiteboard onto tracing paper.

standards-compliant RDF serialization. Export would need an IRI policy. Parallel edges collapse under set semantics unless their identities are modeled separately.

OWL provides an ontology language, with an RDF serialization, for saying what relationships mean (subclasses, domains, inverses, transitivity). A gestaltdb **Ontology** *selects from* OWL a small decidable fragment (`PROFILE = "gestalt-core-v1"`) and *contains* **Axioms** — see *An ontology turns vocabulary into implications*. Intuition: the tracing paper doesn't replace the whiteboard; it is an explicit, lossy-but-uniform copy. The projection is caller-owned. `reasoning/adapters/graph.py` maps current graph records to facts. In contrast, `ontology/adapters/rdf.py` imports selected *schema axioms* from RDF documents; it is not a general instance-data (ABox) loader. Strict import rejects unsupported constructs. Non-strict import explicitly marks the report incomplete and requires separate consent before compilation.

1.6 Stage 3 — the reasoning layer

Axioms *compile to* **Rules** over **Facts**; **Datalog** *defines* that rule shape (safe, function-free Horn rules — see *How a reasoning engine works*). **Prolog** is a related logic-programming language, typically evaluated by goal-directed search with clause order, compound terms, and optional control features. It is not simply Datalog with more keywords. `DatalogEngine` evaluates facts or evidence supplied by callers; explicit adapters can obtain those from current records or temporal claims. The engine does not read a database or construct a `SamplerSnapshot` itself. Passing inferred endpoints as sampler seeds does not make later hops traverse inferred edges.

Cypher and Datalog both bind variables by matching relations, but their contracts differ: Cypher returns query rows (and can perform writes), whereas positive Datalog computes a set-valued closure. Intuition: the engine reads the tracing paper plus the rulebook and pencils in every arrow the rules demand (`alice` is a `Person`), while the whiteboard itself stays untouched.

Parallels: LPG systems (Neo4j/Cypher) keep labels and types as queryable data; RDF/OWL stores (RDFLib, OWL-RL, Jena) treat triples plus entailment regimes as the data model. GestaltDB is an LPG store with an opt-in, offline semantic overlay — nothing is silently rewritten.

1.7 Theory and further reading

- Graph database and Graph query language survey the LPG/RDF split and the language families (Cypher, Gremlin, SPARQL) that grew around each side.
- Cypher is pattern matching over stored structure; SPARQL is the W3C counterpart over triple stores (SPARQL 1.1 overview). GestaltDB deliberately exposes Cypher but no SPARQL endpoint — inferred triples are consumed through explicit adapters, not a second query language.
- Resource Description Framework (RDF 1.1 Concepts) defines the triple model gestaltdb projects onto; the Semantic Web article gives the original vision (machine-readable web of data) that OWL and the ontology layer come from.

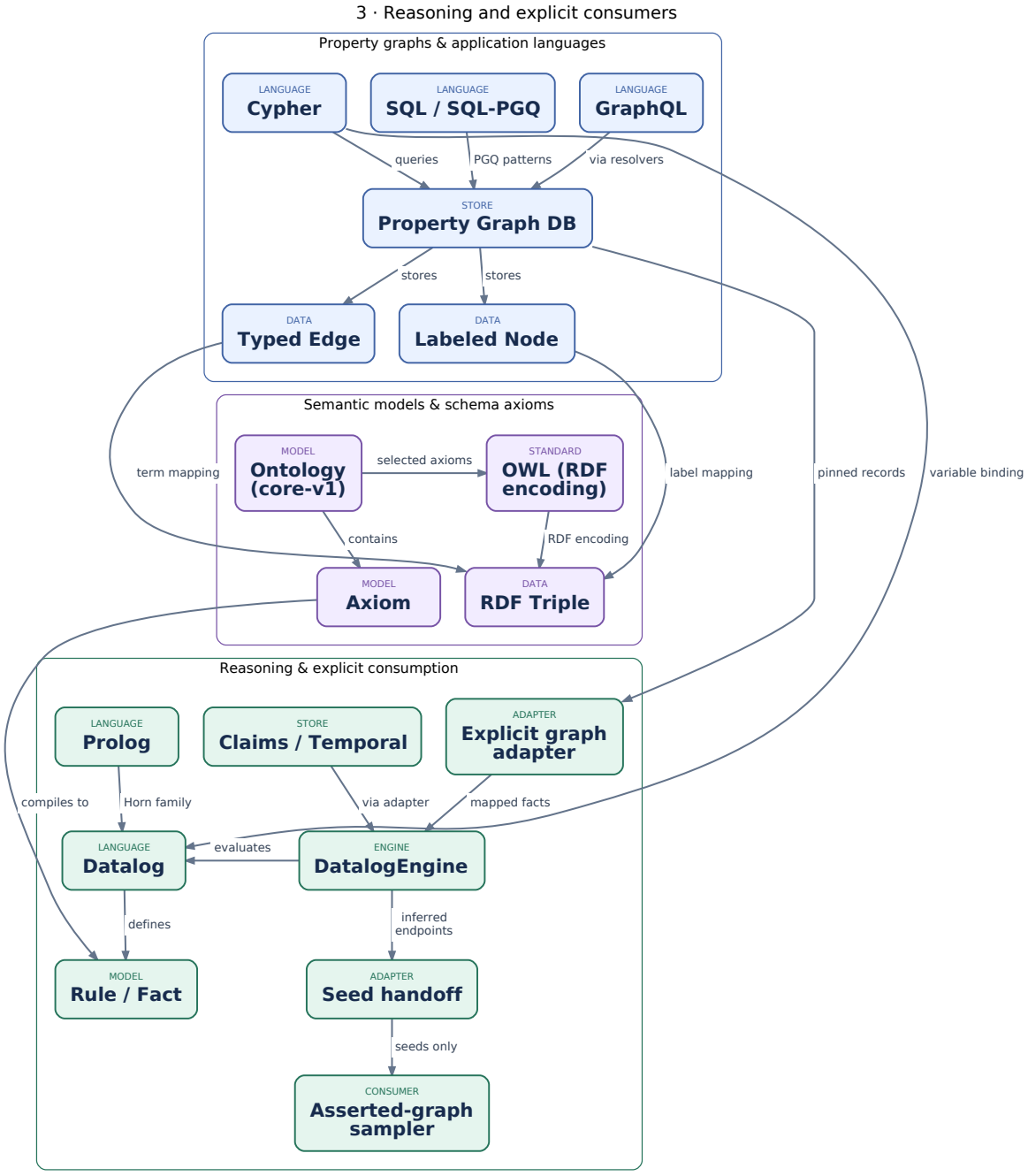


Fig. 3: Stage 3: an assistant that fills in the missing arrows.

- GraphQL specifies client-shaped API selection; its resolvers choose how and where to fetch the data.

```
from gestaltdb.graphdb import Edge, GraphDB, Node
from tempfile import TemporaryDirectory

with TemporaryDirectory() as directory:
    graph = GraphDB.create(directory, backend="lmdb", serializer="json")
    try:
        graph.put_nodes([
            Node(node_id="alice", labels=["Employee"],
                 properties={"name": "Alice"}),
            Node(node_id="acme", labels=["Organization"]),
        ])
        graph.put_edge(Edge(edge_id="e1", source="alice", target="acme",
                            properties={"type": "worksFor"}))
        rows = graph.query(
            'MATCH (a:Employee) RETURN a.name AS name').records
        assert rows == [{"name": "Alice"}]
        assert not graph.query('MATCH (a:Person) RETURN a').records
    finally:
        graph.close()
```

The second assertion is the important one: without an explicit reasoning step, the stored label `Employee` does not imply a stored label `Person`. The first query returns exactly `[{"name": "Alice"}]`; the second returns no rows. No fact about Alice's identity is missing—we have not yet told a reasoner what `Employee` means. Neither label lookup nor employment alone establishes that Alice is a researcher.

1.8 Check your understanding

Question. If Alice's node has label `Employee`, why doesn't a normal `MATCH (a:Person)` return it? Is this a failure of graph storage?

Answer. The query matches a different stored label. Subclass meaning and the decision to consult inferred membership are separate from label lookup. The database is doing the requested lookup correctly. Chapter 2 supplies the meaning; chapter 4 connects it to stored records without changing the labels.

CHAPTER

TWO

AN ONTOLOGY TURNS VOCABULARY INTO IMPLICATIONS

2.1 Why a schema is more than a set of labels

Chapter 1 stored an employee but could not retrieve her as a person. The missing piece is not another index: it is a statement that every employee is a person. An **ontology** makes statements about the modeled vocabulary. Its **axioms** are assumptions the reasoner may use, not observations it discovered itself.

Our two axioms are intentionally small:

- `Domain("worksFor", "Employee")`: the subject of an employment `worksFor` statement belongs to `Employee`.
- `SubClassOf("Employee", "Person")`: every instance of `Employee` also belongs to `Person`. It says nothing about whether every person is a employee; implication is directional. Employment does not establish a job role.

Schema statements like these are often called the **TBox** (terminological knowledge); statements about Alice and Acme are the **ABox** (assertional knowledge). These are useful conceptual roles, not necessarily two physical databases. An RDF graph can contain both kinds of statement.

2.2 Reading the notation

`(alice, worksFor, acme)` is a triple. `(x, rdf:type, Employee)` is a pattern with a variable `x`. When `x = alice`, the pattern becomes a ground statement: there are no variables left. In formulas, `rdf:type` abbreviates the full reserved IRI supplied by `TYPE_PREDICATE` in Python.

`C ⊑ D` reads "class C is a subclass of class D." An inference bar reads "if the statements above the line hold, conclude the statement below it":

$$\frac{(x, \texttt{rdf:type}, C)}{(x, \texttt{rdf:type}, D)}\ (C \sqsubseteq D) \qquad \frac{(x, p, y)}{(x, q, y)}\ (p \sqsubseteq q) \qquad \frac{(x, p, y)\ \ (y, p, z)}{(x, p, z)}\ (\text{Transitive}(p))$$

Read the three cases as membership inheritance, property inclusion, and transitivity. In the last case, joining `x → y` to `y → z` on the shared `y` permits `x → z` *only because p is declared transitive*. The mere existence of a two-hop path does not make every relationship transitive.

2.3 From one assertion to two consequences

Intuition. Read the picture below in two groups: blue is *data* (what someone asserted), purple is *schema* (what the arrows mean). `domain` says "anything a `worksFor` arrow leaves is an `Employee`"; `subClassOf` says "every `Employee` is also a `Person`". The `inferred type` arrows are computed consequences, not stored labels. Both `Employee` and `Person` memberships follow from the one asserted `worksFor` fact:

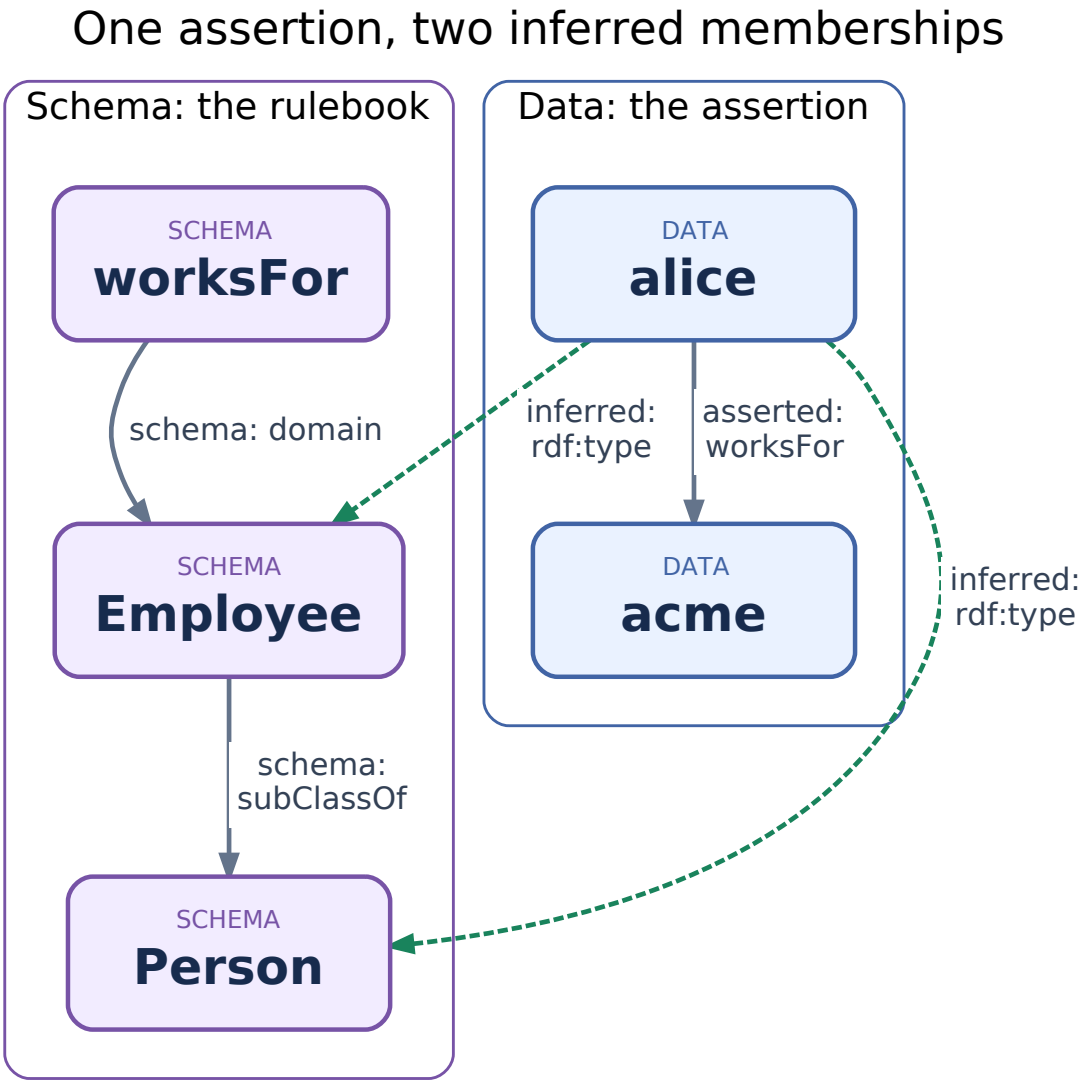


Fig. 1: Schema and data are visually separate; memberships are computed in this example, not written back. Explicit publication is a separate API.

```
from gestaltdb.ontology import Ontology, SubClassOf, Domain,␣
↪compile_ontology
from gestaltdb.reasoning import DatalogEngine, Fact, ReasoningLimits
from gestaltdb.ontology import TYPE_PREDICATE
```

(continues on next page)

(continued from previous page)

```
plan = compile_ontology(Ontology("people", [
    Domain("worksFor", "Employee"),
    SubClassOf("Employee", "Person"),
]))
asserted = Fact("alice", "worksFor", "acme")
result = DatalogEngine().evaluate(
    plan.program, [asserted],
    limits=ReasoningLimits(), collect_proofs=True)
employee = Fact("alice", TYPE_PREDICATE, "Employee")
person = Fact("alice", TYPE_PREDICATE, "Person")
assert result.facts == tuple(sorted((asserted, employee, person)))
assert Fact("alice", TYPE_PREDICATE, "Researcher") not in result
assert not result.explain(person).truncated
```

Read this as a two-step explanation: the domain rule derives `Employee`; the subclass rule uses it to derive `Person`. `result` retains the asserted fact as well as both conclusions. Its explanation is one bounded witness, not a complete inventory of alternative supports for deletion maintenance.

> **Note**
>
> Domain is an implication, not an input-validation constraint. An untyped subject of `worksFor` gains `Employee` membership; it is not rejected for lacking that label. Short names in this example are exact local symbols: `Person` and `person` are different constants, and only `TYPE_PREDICATE` supplies the full reserved `rdf:type` IRI.

2.4 Refinement needs evidence about the job

Calling every employee a researcher would encode a false assumption as an axiom. Instead, place `Researcher` *below* `Employee` in the class hierarchy. This states that researchers are employees in this model; it does not classify all employees as researchers. We use `Researcher` here as an employment-role class, not a claim that all research activity requires employment.

Now introduce a separate relation, `hasJobDescription`. For this exercise its object is the named category `researcher` or `not_researcher`. These are entity symbols in the binary engine, not datatype literals, and the lowercase category `researcher` is not the class `Researcher`. To classify an employee, we need a rule that joins *their* membership with *their* research-job evidence:

$$\text{type}(x, \text{Employee}) \wedge \text{hasJobDescription}(x, \text{researcher}) \Rightarrow \text{type}(x, \text{Researcher})$$

This value-sensitive condition is not a plain domain axiom: the domain would apply to **every** job description, including `not_researcher`. The selected ontology compiler does not offer an arbitrary OWL value restriction for this case, so we add an explicit safe binary rule. Chapter 3 explains its notation and evaluation in depth.

```
from gestaltdb.reasoning import Atom, Program, Rule, Variable

refined = compile_ontology(Ontology("employment-roles", [
```

(continues on next page)

(continued from previous page)

```
    *plan.ontology.axioms, SubClassOf("Researcher", "Employee"),
]))
x = Variable("x")
research_job = Rule("employee-with-research-job", [
    Atom(x, TYPE_PREDICATE, "Employee"),
    Atom(x, "hasJobDescription", "researcher"),
], Atom(x, TYPE_PREDICATE, "Researcher"))
program = Program([*refined.program.rules, research_job])
engine = DatalogEngine()
researcher = Fact("alice", TYPE_PREDICATE, "Researcher")
assert researcher not in engine.evaluate(program, [asserted],␣
↪limits=ReasoningLimits())
with_job = engine.evaluate(program, [
    asserted, Fact("alice", "hasJobDescription", "researcher"),
], limits=ReasoningLimits(), collect_proofs=True)
assert all(fact in with_job for fact in (employee, person, researcher))
other_job = engine.evaluate(program, [
    asserted, Fact("alice", "hasJobDescription", "not_researcher"),
], limits=ReasoningLimits())
assert employee in other_job and researcher not in other_job
job_alone = engine.evaluate(program, [
    Fact("alice", "hasJobDescription", "researcher"),
], limits=ReasoningLimits())
assert employee not in job_alone and researcher not in job_alone
```

The last case matters: a category record alone is not evidence of an actual employment relationship. Equally, no description at all is not evidence for the `not_researcher` category. That category is an explicit positive classification in this exercise, not logical negation. The program does not enforce that only one category may be recorded; contradictory or overlapping descriptions need a separately stated validation policy.

2.5 How much of the domain have we actually defined?

"An employee is a person who participates in gainful employment" is a richer conceptual definition than `Employee` subclass `Person`. A fuller model could give an employment activity its own identity, connect the person and employer to that activity, and attach the job description, pay, and validity to it. This also handles one person holding different roles at different companies.

`Employee` subclass `Person` is therefore a necessary-condition statement, not a complete if-and-only-if definition of gainful employment. Deriving the class membership does not construct an employment activity, a company, or a job-description record. The direct `hasJobDescription` relation is a simplification for one current job; in the richer model the description would belong to a particular employment activity rather than globally to the person.

The examples stop short of that reification. They treat `worksFor` as supplied evidence of employment, rather than inferring gainful activity from physical co-location. Visiting or working inside a company's building is not sufficient by itself. Likewise, typing ACME as a `Company` requires an assertion or an explicit range rule; its name does not supply that fact. These are deliberate modeling boundaries, not facts a reasoner can guess.

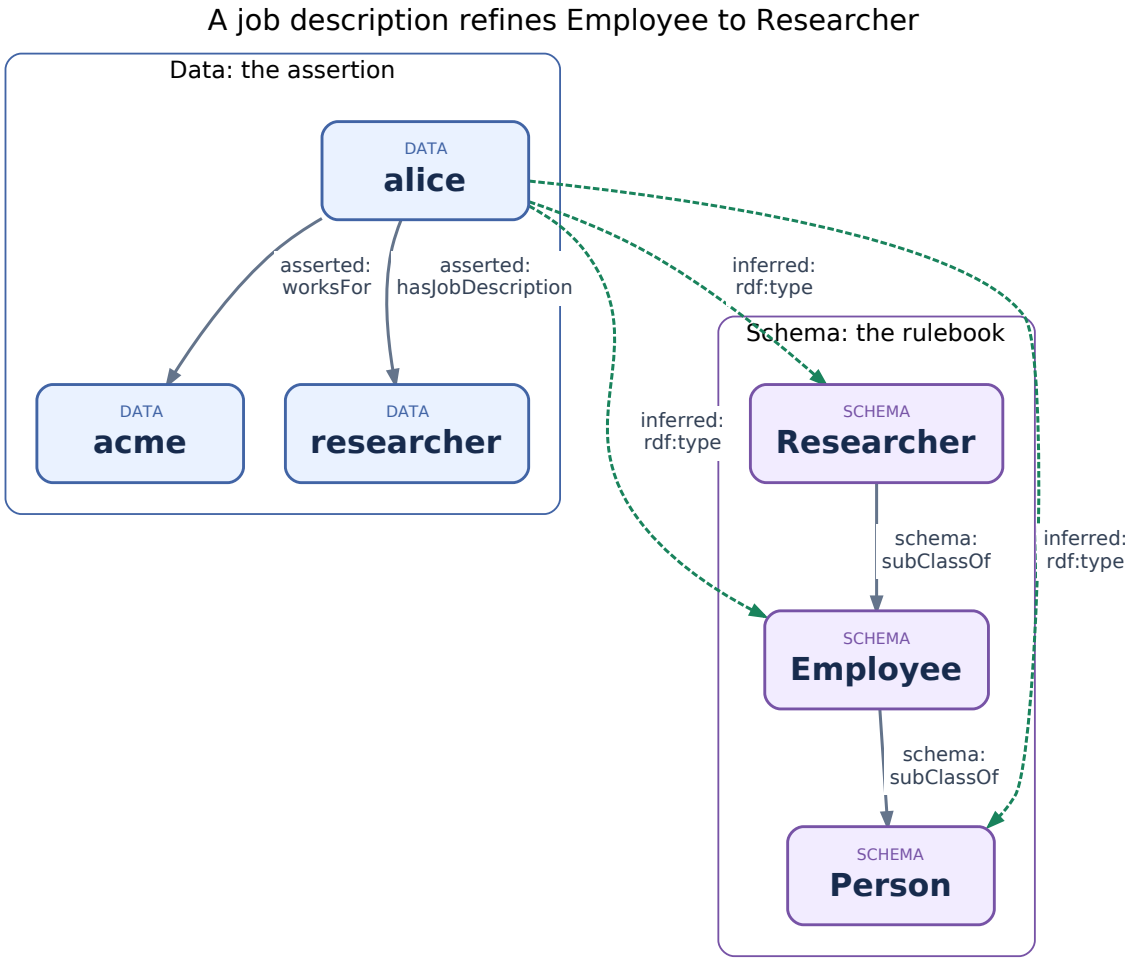


Fig. 2: A further refinement: the Researcher membership now depends on an additional asserted job description, not merely on `worksFor`.

2.6 What compilation contributes

The ontology is convenient for expressing domain meaning; the evaluator needs executable rules. `compile_ontology` is the bridge. It rewrites the two axioms into "if worksFor, infer Employee" and "if Employee, infer Person," retaining an origin mapping from generated rules back to source axioms. That mapping is useful when explaining *which modeling assumption* produced an answer.

The selected `gestalt-core-v1` fragment includes subclass/subproperty, equivalence, domain/range, inverse, symmetric and transitive properties, binary property chains, selected left-side class expressions, and scoped reflexivity. It is not full RDFS or OWL. Disjunctive conclusions, arbitrary anonymous restrictions, longer property chains, and unscoped reflexivity must not be inferred from the fact that the engine supports simpler forms. See *Choosing a supported reasoning workflow* for boundaries and the library's ontology reference for API details.

2.7 Theory and further reading

- OWL 2 overview introduces the wider ontology language. Supporting some OWL-shaped axioms is not the same as implementing every OWL profile.
- Description logic provides the foundation for much ontology reasoning, including languages with richer class expressions than this tutorial's named-class fragment.
- Horn clause explains the single-conclusion rule shape used by this compiler. The next chapter explains how an engine evaluates such rules, rather than treating them as magic.

2.8 Check your understanding

Question. If we remove the domain axiom, does `SubClassOf(Employee, Person)` alone still imply that Alice is a person from the `worksFor` fact? What if we instead assert Alice's `Employee` membership directly? Does either case establish that her job is research?

Answer. The first inference no longer follows: nothing supplies the `Employee` premise. The second does follow, because the asserted membership is enough to trigger the subclass implication. A rule does not manufacture its own missing premise. Neither case establishes a research job without the additional job evidence and classification rule.

CHAPTER

THREE

HOW A REASONING ENGINE WORKS

3.1 Why not just write another Python loop?

For two implications, a hand-written `if` statement seems sufficient. The problem appears when one conclusion becomes another rule's input, a hierarchy gets deeper, several applications share the same policy, or an answer needs an explanation. A loop can implement all this, but then *you* must decide which joins to repeat, suppress duplicates, terminate recursion, and track why each answer exists. Those are the responsibilities a reasoning engine makes reusable.

A rule separates a statement of intended meaning from an execution strategy. The same subclass rule can classify people from many datasets; the engine decides which facts to inspect and how to reuse results. The benefit is not merely less code. Search can include indirectly classified entities, dependency analysis can discover effects several steps away, and an explanation can name the assumptions behind a result. The cost is additional computation, memory, and responsibility for the validity of the rules.

This chapter introduces one important family: **finite positive Datalog**. It is enough for the selected ontology compiler, but it is not a synonym for every kind of reasoning. An engine for arbitrary OWL class expressions, an answer-set solver, and a Prolog interpreter need not accept the same language or compute the same kind of answer.

3.2 From a sentence to a rule

Write `worksFor(alice, acme)` as a convenient relation notation for the triple `(alice, worksFor, acme)`. `worksFor` is the **predicate**, and `alice` and `acme` are **constants** denoting particular entities. An **atom** such as `worksFor(x, y)` may contain variables. A **fact** is a ground atom: no variables remain to be assigned.

Now suppose the graph also states that Acme is located in Athens. We want to derive Alice's work location, under the explicit simplifying assumption that a employment relationship inherits its organization's location:

$$\text{worksFor}(x, y) \wedge \text{locatedIn}(y, z) \;\Rightarrow\; \text{worksIn}(x, z)$$

The left side is the **body**, the right side the **head**. The symbol ∧ means "and"; ⇒ means "implies." All variables are implicitly universally quantified: *for every* assignment to `x`, `y`, and `z`, if both premises hold, the conclusion holds. Datalog texts often reverse the typography:

```
worksIn(x, z) :- worksFor(x, y), locatedIn(y, z).
```

Read `:-` as "if" and the comma as conjunction. This is explanatory notation, not a textual parser exposed by the GestaltDB examples below. Python code uses `Rule` and `Atom` values. A `Variable("x")` denotes a variable; the plain string `"x"` remains a constant.

3.3 Matching a rule is a join

The repeated `y` is the important part. It requires *the same organization* in both premises. In database language this is an equijoin. A fact about Bob working for a different organization must not combine with Acme's location merely because the two relations are nonempty.

The assignment `σ = {x ↦ alice, y ↦ acme, z ↦ athens}` is a **substitution**. Applying it to the body gives two ground premises that can be checked against the known facts; applying it to the head gives `worksIn(alice, athens)`. With a large dataset, finding these assignments efficiently is most of the work. Binding `y` to Acme before looking up `locatedIn` avoids scanning every organization's location for every employee.

A rule is **safe** here when every head variable also occurs in its positive body. `worksIn(x, z) :- worksFor(x, y)` would leave `z` unconstrained. It does not specify which place to invent, so this engine rejects it rather than guessing. Safety is one reason the supported rule language is deliberately smaller than unrestricted first-order logic.

3.4 Why recursion needs a fixed point

One join answers a two-relation question. It does not by itself answer "Which larger organizations contain Acme, however deep the hierarchy?" For that, let `partOf` be transitive:

$$\text{partOf}(x, y) \wedge \text{partOf}(y, z) \;\Rightarrow\; \text{partOf}(x, z)$$

Acme may be part of a group, the group part of a university, and the university part of a federation. Newly discovered containment facts can be used by the same rule again. This is recursion over a relation, not necessarily a recursive Python function. A single pass in a fixed order is not a general solution.

An engine accumulates facts until another round adds nothing. Define the input set as `F`, the rules as `P`, and the accumulated set after round `n` as `I_n`:

$$I_0 = F, \qquad I_{n+1} = I_n \cup \{\, h\sigma \mid (B \Rightarrow h) \in P,\ B\sigma \subseteq I_n \,\}, \qquad C_P(F) = \bigcup_{n \geq 0} I_n$$

`Bσ ⊆ I_n` says that *every* substituted body atom is already known. The union retains old facts while adding conclusions. `C_P(F)` is the **closure**: all assertions plus their consequences in this fragment. For chapter 2, the initial set contains `worksFor(alice, acme)`, the next adds Employee membership, the next adds Person membership, and the following adds nothing.

This is a **least fixed point**: the smallest closed set containing the input, not a collection of arbitrary additional truths. With finitely many constants, fixed predicate arities, and no operations that generate fresh values, only finitely many distinct facts can appear. Set semantics prevents repeated derivations from growing the set forever. Cyclic rules can rediscover facts; they do not create an infinite list of duplicate answers.

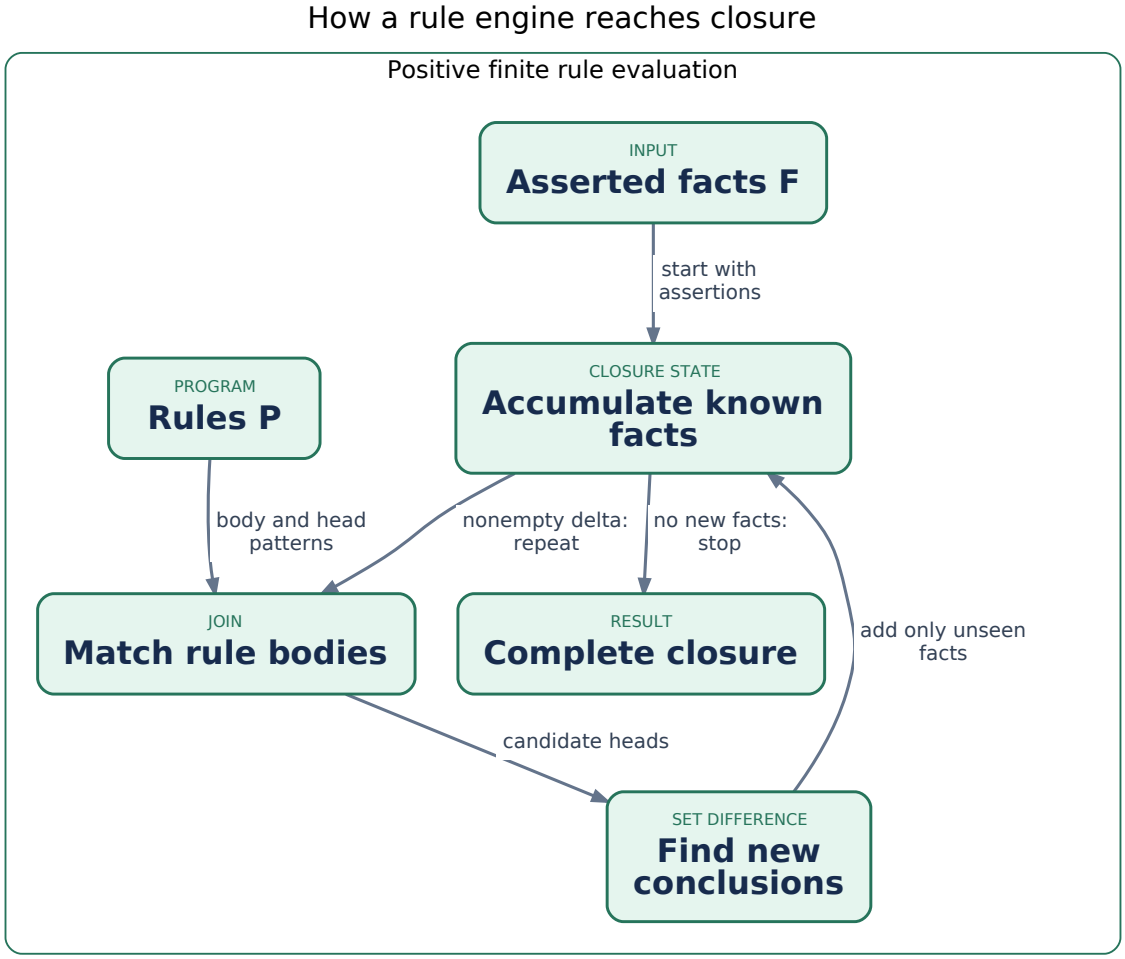


Fig. 1: The evaluation loop. Begin with the assertions, repeat only while new facts are found, and report complete closure only after the stopping condition.

Note

The termination argument is for this finite positive fragment. Arithmetic that generates unbounded values or existential rules that create fresh individuals need different guarantees. "It looks like Datalog" is not a proof of termination for every language extension.

3.5 Try a join and a recursive rule

The following program expresses both operations directly. It does not need an ontology compiler or database backend, demonstrating that a reasoning engine and a graph database are separable components.

```
from gestaltdb.reasoning import (
    Atom, DatalogEngine, Fact, Program, ReasoningLimits, Rule, Variable,
)

x, y, z = Variable("x"), Variable("y"), Variable("z")
program = Program([
    Rule("work-location", [Atom(x, "worksFor", y), Atom(y, "locatedIn",␣
↪z)],
         Atom(x, "worksIn", z)),
    Rule("containment", [Atom(x, "partOf", y), Atom(y, "partOf", z)],
         Atom(x, "partOf", z)),
])
facts = [
    Fact("alice", "worksFor", "acme"),
    Fact("acme", "locatedIn", "athens"),
    Fact("acme", "partOf", "group"),
    Fact("group", "partOf", "university"),
    Fact("university", "partOf", "federation"),
]
result = DatalogEngine().evaluate(
    program, facts, limits=ReasoningLimits(), collect_proofs=True)
assert result.match(predicate="worksIn") == (
    Fact("alice", "worksIn", "athens"),)
assert Fact("acme", "partOf", "federation") in result
assert not result.explain(Fact("acme", "partOf", "federation")).truncated
```

3.6 What makes the answer trustworthy?

The notation $F \cup P \models q$ reads "the assertions and rules entail q": every interpretation satisfying those assumptions also satisfies the conclusion. The engine computes consequences; it does not establish that the assertions or modeling assumptions describe the real world correctly. Our work-location rule, for example, is unsuitable for remote workers unless the model accounts for them.

An evaluator is **sound** for a fragment if every answer it produces follows under that fragment's semantics. It is **complete** for a fragment if it can produce every required answer. These qualifications matter: completeness for safe positive binary rules is not completeness for OWL DL or arbitrary Python conditions. Resource

limits are also distinct from semantics. A limit failure must not masquerade as a complete successful closure.

An explanation provides another kind of assurance: it lets a reader inspect the actual premises and rule used. That is why the examples collect proofs. One chosen witness is useful for auditing an answer, but it is not all possible supports. If a premise is deleted, a second independent derivation may still justify the answer. Chapter 5 returns to that distinction.

3.7 Avoiding unnecessary work

A naive implementation repeats all joins after every round. **Semi-naive** evaluation instead uses the latest additions, called the delta Δ, in at least one body occurrence of each newly relevant join. The other occurrences may use the accumulated facts. Joining only new facts with new facts would be incorrect: a new employment fact must still join an old location fact. Subtracting already-known conclusions gives the next delta.

Not every rule needs to run in one global loop. An engine can form a dependency graph of predicates and find **strongly connected components** (SCCs): groups that can recursively depend on themselves. Prepared plans use such groups to schedule recursion and reuse dispatch information. This is the same general idea as identifying loops in a compiler's control-flow analysis. Planning does not change the intended answers, and plan construction itself costs time.

Full closure is useful when many later queries reuse its results. If the only question is about Alice, deriving every person's work location may be wasted effort. A **goal-directed** strategy starts from the question. GestaltDB's demand API uses magic-set specialization for its supported positive fragment: bindings from the goal restrict which derivations are needed. That is not a promise that every query becomes faster, nor that storage extraction is always pushed down to the smallest possible KV read.

3.8 Why intermediate representations exist

A domain author wants to write `SubClassOf`; a join operator wants indexed relations or integer columns. These are different views of the same task. An **intermediate representation** (IR) makes each boundary explicit. The ontology compiler produces `Program`/`Rule`/`Atom` values; a prepared plan adds an execution schedule; a compact batch stores relation rows in columns. Inspecting these layers explains both answers and costs without requiring one representation to serve every purpose.

Replacing the repeated symbol `alice` with an integer can reduce payload and comparison costs, but that integer has meaning only with its exact dictionary. Two snapshots may assign different numbers to Alice. This is why dictionary and context identity are part of compact-batch contracts, not optional metadata. Arrow can exchange those columns; Polars can execute selected columnar joins. Neither transport nor compactness supplies logical semantics, proofs, or maintenance automatically. The default binary evaluator remains indexed Python; the Polars path is explicitly selected and closure-only.

To inspect these boundaries in GestaltDB, see the library references for plans, batches, and demand evaluation. The typed n-ary engine is a separate API, not an implicit upgrade of the binary ontology program used here.

3.9 Other engines: similar notation, different jobs

Soufflé is a useful comparison for bottom-up Datalog and recursive program analysis. It can compile programs to native executables and emphasizes relation processing and profiling. Its practical language has extensions beyond finite pure Datalog; its own tutorial explains why value-generating arithmetic can break termination. The connection to GestaltDB is the relational/fixpoint model, not an identical API, performance profile, or language contract.

SWI-Prolog's tabled execution illustrates a different route to recursive answers: goal-directed evaluation with memoized answers and suspended recursive calls. Ordinary depth-first Prolog search can loop on some recursive definitions; tabling changes that behavior for appropriate programs. It would be misleading to describe all Prolog engines as naive depth-first search, or all goal-directed reasoning as GestaltDB's magic-set strategy.

Clingo combines a grounder and an answer-set solver. It is relevant when the problem includes choices, constraints, and alternative solutions, such as allocating resources under restrictions. Such a program can have no stable model or several; that is not the same output contract as one positive Datalog closure. GestaltDB uses Clingo in selected comparative benchmarks, not as the default runtime or as proof of equivalent support for answer-set programming.

Apache Jena's inference API shows how an RDF system can expose an inference model through which queries see deductions as well as base statements. Jena documents different reasoners, rule-engine strategies, and coverage limits. GestaltDB instead makes its database-to-reasoner and result-consumption boundaries explicit. The comparison helps explain an integration choice, not a claim that either system implements every OWL construct.

The practical lesson is to choose an engine by **the questions and semantics required**, then assess execution and integration. Similar-looking rule syntax does not establish interchangeability. None of these external engines is a required dependency for the examples in this chapter.

3.10 Check your understanding

Question. Why must a new `worksFor(alice, acme)` fact join an *old* `locatedIn(acme, athens)` fact? Would a query about Alice require computing every organization's transitive closure?

Answer. The body is satisfied regardless of the premises' arrival order; semi-naive evaluation therefore combines a delta occurrence with accumulated relations. A supported demand strategy can restrict work relevant to Alice, while a full materialization deliberately computes reusable closure. Those are execution choices, not different meanings of the positive rule.

3.11 Theory behind the explanations

Datalog gives the language background. The least fixed point and Knaster–Tarski theorem explain why monotone consequence operators have least fixed points; finiteness additionally makes the iteration above terminate. Magic sets gives the rewriting background for selective evaluation. These are foundations, not speedup guarantees.

CHAPTER

FOUR

CONNECTING A DATABASE TO A REASONER

4.1 Why the engine should not read an arbitrary live graph

Chapter 3 reasoned over a Python list of facts. Real applications do not start with that list: they start with stored nodes, labels, properties, and edges. The first integration problem is deciding which stored fields mean which logical predicates. An edge called `worksFor` is an application convention; the engine should not guess that every string property is an entity relation.

The second problem is consistency. Suppose an appointment is read before a correction but its related organization is read afterward. The assembled input could describe no state that ever existed. Correct rule evaluation would then produce a logically consistent answer about an inconsistent extraction. Pinning a read snapshot prevents that kind of moving target where the backend supports the required guarantee.

GestaltDB therefore makes the boundary explicit. A mapping selects fields; a read view pins the source; an adapter supplies logical facts and evidence; the evaluator computes consequences. The evaluator itself stays independent of storage, so the same program can be tested over a small list or evaluated over an extracted graph without changing its meaning.

4.2 A complete stored-graph example

We now deliberately store Alice **without** an `Employee` or `Person` label. The only relevant assertion is the appointment edge. The ontology from chapter 2 will supply both memberships. Follow the four steps in order: store, map, evaluate, and compare the logical answer with the unchanged stored graph.

```
from tempfile import TemporaryDirectory
from gestaltdb.graphdb import Edge, GraphDB, Node
from gestaltdb.ontology import (
    Domain, Ontology, SubClassOf, TYPE_PREDICATE, compile_ontology,
)
from gestaltdb.reasoning import Fact, ReasoningLimits
from gestaltdb.reasoning.adapters.graph import GraphMapping,␣
↪PinnedGraphSource

compiled = compile_ontology(Ontology("employment", [
```

(continues on next page)

(continued from previous page)

```
    Domain("worksFor", "Employee"), SubClassOf("Employee", "Person"),
]))
with TemporaryDirectory() as directory:
    graph = GraphDB.create(directory, backend="lmdb", serializer="json")
    try:
        graph.put_nodes([Node("alice"), Node("acme")])
        graph.put_edge(Edge("appointment", "alice", "acme", {"type":
↪"worksFor"}))
        mapping = GraphMapping(edge_types=[("worksFor", "worksFor")])
        with graph.current_read_view(require_verifiable=True) as view:
            with PinnedGraphSource(view, mapping=mapping) as source:
                logical = source.evaluate(
                    compiled, limits=ReasoningLimits(),␣
↪collect_proofs=True)
        person = Fact("alice", TYPE_PREDICATE, "Person")
        assert logical.match(subject="alice", predicate=TYPE_PREDICATE) ==␣
↪tuple(sorted([
            person, Fact("alice", TYPE_PREDICATE, "Employee"),
        ]))
        assert not logical.match(subject="alice", predicate=TYPE_PREDICATE,
                                 object="Researcher")
        assert not logical.result.explain(person).truncated
        assert graph.get_node(b"alice").labels == ()
        assert not graph.query("MATCH (a:Person) RETURN a").records
    finally:
        graph.close()
```

The mapping's first `worksFor` is a physical edge type; the second is the logical predicate. They happen to have the same spelling, but need not. This is an allowlist: unmapped fields are ignored. A label-to-class mapping could also supply membership assertions. Projection uses set-valued facts, so several physical edges can support the same logical statement without multiplying that statement in the closure.

The two membership assertions succeed, while the label query still returns no rows. That is the point of the exercise. An in-memory reasoning result is not a hidden write to the database. The immutable result may outlive the read view; extraction must finish while the view remains open. No Researcher membership follows from employment alone; the additional rule in chapter 2 would also need a mapped job-description fact. Notebook 3 adds a visualization of this caller-owned logical result.

4.3 When results should become durable

An application might reuse the inferred memberships in many reports. It could keep the result in memory, recompute it, or explicitly publish a materialized generation. Publication is a different operation from evaluation: it creates persisted derived claims with a source context and a lifecycle.

That separation matters when writers are active. Imagine computing Alice's membership, then publishing it after the appointment was retracted. A staged publication must compare its source/target identities with the state it is allowed to publish against; a stale stage needs an explicit retry rather than quietly becoming current.

Recovery after an interrupted nontransactional write also requires an audit policy, not an assumption that every artifact is valid. See the library's generation and publication guides for the point-publication contracts; they are not implicit behavior of `evaluate`.

4.4 Why extraction can dominate performance

An excellent join plan cannot recover time already spent decoding every record or authenticating a snapshot. Bounded extraction batches limit an interface's batch size, not necessarily total process memory or the authentication cost. Native KV scans can reduce some overhead, but raw bytes still need decoding and visibility checks before they become evidence. Backend capabilities differ; an adapter must reject an unsupported requested guarantee rather than quietly substitute a weaker read.

For this reason the example chooses LMDB explicitly. It does not assume that every backend offers the same verifiable current snapshot. The deeper APIs are documented in the library's graph adapter and extraction guides.

4.5 Check your understanding

Question. The logical result contains Alice's Person membership. Why does the final Cypher label query still return nothing? Could we safely re-run extraction after closing the read view?

Answer. The result was not published as a stored label, and plain Cypher does not automatically consult it. Extraction borrows the open pinned view; only the completed immutable result can be used independently afterward.

CHAPTER

FIVE

CLAIMS, TIME, AND BELIEF

5.1 Why a triple is sometimes not enough

So far, `worksFor(alice, acme)` has been an unqualified assertion. A research directory eventually needs to ask *who supplied it*, *when the appointment was valid*, and *what was known when a report was produced*. Attaching a date to a node's display properties does not automatically answer those questions.

Consider an appointment valid from January through March. The database first records it in February, then receives a correction in April saying it actually ended in February. A report generated in March and a corrected report generated in May may answer the same question about March differently. **Valid time** describes the modeled world; **system time** describes which recorded history the application is allowed to consult. Their separation is called bitemporality. It supports historical audit and avoids presenting later corrections as if they had been known earlier.

GestaltDB keeps mutable graph records, immutable temporal versions, and sourced claims as separate layers. `put_node_version` and `commit_versions` append history; as-of reads select it. `assert_claim`, `correct_claim`, and `retract_claim` record sourced propositions through that history. These are different workflows from changing a current node's properties.

5.2 Missing evidence is not a negative assertion

Suppose one source affirms Alice's appointment and another explicitly disputes it. Removing one physical edge would conceal the disagreement. A claim layer can retain both pieces of evidence. `claim_status` distinguishes `supported`, `refuted`, `both`, and `unknown`. These are evidence states, not four confidence scores: `unknown` means neither positive nor negative evidence was found in the selected context, while `both` records a conflict.

This explains why absence must be treated carefully. If the directory lacks Bob's appointment, it has not established that Bob has no appointment. To reason from absence, an application must declare an appropriate closed-world scope, such as an authoritative and complete roster for a particular horizon. The generic engine's scoped stratified negation is a separate facility from the positive binary programs used earlier. It cannot be enabled by simply interpreting every missing fact as false.

5.3 Combining time with derivations

If a conclusion needs two premises, it holds only where both premises are valid. If either of two independent derivations is enough, their valid intervals offer alternative support. This motivates intersection *within* a derivation and union *across* derivations. Using half-open `[start, end)` intervals makes boundary behavior explicit: the ending instant belongs to the next interval, not to both.

$$V_s(h) = \bigcup_{d \in D_s(h)} \bigcap_{p \in \text{premises}(d)} V_p(h), \qquad \text{holdsThroughout}(s, W, h) \iff W \subseteq V_s(h)$$

At a pinned system horizon `h`, `D_s(h)` denotes the admissible positive supports for statement `s`. For this positive-rule illustration, a derivation is valid where *all* premises overlap; alternative derivations contribute a *union*. An assertion supplies its selected valid interval directly. This is not the definition of every window-query policy or of negation/equality. For example, supports covering `[1,2)` and `[2,3)` together cover `[1,3)`, even though neither support alone does. `[1,2)` and `[3,4)` leave a real gap; replacing them by their min/max envelope would invent truth at time 2.5. Truth coverage may coalesce while evidence history keeps both supports.

Temporal closure therefore evaluates supported change segments, not just a sample at one instant. See the library's temporal reasoning guide for the supported contracts. It is a stronger task than attaching a timestamp to a point answer.

5.4 Why deleting an assertion can be difficult

Suppose Alice's Person membership follows both from the appointment and from an independently sourced Employee assertion. Deleting the appointment must not remove the still-supported Person membership. Conversely, a cycle of rules must not keep an answer alive forever after its final real support disappears. This is the problem of **truth maintenance**: preserving exactly the conclusions still justified after changes, not counting an arbitrary number of proof paths.

GestaltDB's positive binary maintenance API tracks bounded supports and uses DRed-style deletion—overdelete potentially affected consequences, then rederive those still justified. Full rematerialization is the explicit fallback for unsupported changes or combinations. A chosen explanation witness alone is not sufficient state for safe deletion. The detailed supported boundaries are in the library's maintenance guide.

5.5 Belief and confidence answer different questions

Finally, "Alice works for Acme" and "agent A believes Alice works for Acme" are different propositions. Modal evaluation uses worlds linked by declared accessibility relations; `entails` evaluates bounded formulas in that frame. An accessibility link is not a physical social-network edge unless the application explicitly models it that way.

Neither a source name, multiple proofs, nor a belief operator automatically produces a probability. Confidence calculi need separately chosen policies; the default rule evaluator does not assign probabilistic truth. Similarly, domain/range

rules derive memberships, while validation reports selected conflicts. These services should not silently replace one another.

5.6 Theory and further reading

- Temporal database distinguishes valid time (when the world was so) from transaction/system time (when the database recorded it). GestaltDB keeps exactly this split: valid intervals on versions and a selected system horizon on reads. `verify_closure` checks supported range results against independently recomputed change segments; it is not a general database-recovery verifier.
- Allen's interval algebra catalogues the relations between time intervals (before, meets, overlaps, …). The tutorial uses concrete half-open `[start, end)` intervals: touching intervals (`[1,2)`, `[2,3)`) share no instant. Allen's qualitative relations are useful background, not a claim that this module implements that algebra.
- Open-world assumption vs closed-world assumption: claims default to open-world (absent evidence means `unknown`, not false); only explicitly `closed` scopes license negation-as-absence (ch03).
- Kripke semantics (possible worlds linked by accessibility relations) is the model behind the modal/belief layer: each agent's frame is a set of worlds, and `entails` checks the formula in the worlds reachable under the depth/state bounds.

5.7 Check your understanding

Question. Bob has no appointment claim in our selected snapshot. Is his appointment refuted? If we later remove one of two independent supports for Alice's Person membership, must that membership disappear?

Answer. No to both. Missing evidence gives `unknown` under the open-world reading; an explicit refutation or an authorized absence-based policy is different. Alice remains supported while an independent valid derivation survives in the selected context.

Losing the last derivation of Person membership would mean that this model no longer supplies that conclusion. It would not assert that Alice has stopped being a person. That difference is exactly why an open-world interpretation and a careful choice of domain axioms matter.

CHAPTER

SIX

QUERYING AND SAMPLING WITH INFERRED FACTS

6.1 Which graph does an answer come from?

We can now answer whether Alice is a person, but the application still needs to choose *which view it is asking*. Chapter 4 deliberately produced two different results: the logical closure contained Person membership, while the stored-label query did not. Neither answer was a bug. They answered questions about different representations.

For a stored-graph question, use Cypher to match physical records, labels, and paths. For a consequence under the chosen ontology, query the completed logical result, such as `logical.match(subject="alice", predicate=TYPE_PREDICATE)` in chapter 4. For a persisted derived view, explicitly publish results through the supported publication workflow. These are three distinct choices; evaluating a program does not secretly make future Cypher queries inference-aware.

This matters especially for counts. Two employment records can have separate edge IDs, one logical proposition, several independent supports, and many paths through a class hierarchy. Counting edges, facts, supports, or paths can therefore give different numbers. Choose the unit of your question before choosing a query API.

6.2 Why sampling is a different operation

A directory endpoint may need every person satisfying a predicate. A graph learning pipeline may instead need a bounded neighborhood around selected entities. Sampling exchanges complete enumeration for a selection policy and a fanout budget. It must state whether the population consists of physical edges, distinct endpoints, or paths: a high-degree endpoint with many proofs should not acquire a larger weight accidentally.

GestaltDB's explicit logical adapters can inspect inferred relations and sample distinct endpoints. `expand_edge_types` identifies mapped relation members; `sample_expanded_neighbors` accounts for their contributions under a total fanout cap. For a seed `s`, let `N_r(s)` be the endpoint set for member relation `r` and `k_r` its quota:

$$N_K(s) = \text{distinct}\left(\bigcup_{r \in R_{exp}} \text{sample}(N_r(s), k_r) \right), \qquad \sum_r k_r = K, \quad |N_K(s)| \le K$$

For `sample_expanded_neighbors`, `N_r(s)` is the distinct logical endpoint set for an expanded member relation, not a bucket of physical edge IDs. Quotas split `K` evenly in sorted relation order, with earlier members taking the remainder. Overlapping endpoints are attributed to the first member; unused quota is not redistributed. Thus fanout 10 is a cap, not a guarantee of ten distinct endpoints or uniform sampling over the union. Multiple proof paths do not increase an endpoint's weight. Inferred seed handoff leaves the next asserted-graph sampling hop unchanged.

The array-native `SamplerSnapshot`/`SamplerEngine` serves repeated training workloads. It trades repeated KV reads for reusable compact arrays and relation indexes. Those arrays are a different artifact from a logical closure. Passing inferred endpoint IDs as seeds selects *where to start* in the snapshot; it does not add inferred edges to the snapshot or change what later hops traverse. External entity IDs also differ from compact snapshot integers and local batch rows. Use the documented mappings rather than joining them by numerical value.

There is a similar trap with filtering. Choosing Alice as an eligible seed does not ensure every intermediate node belongs to a desired cohort. To constrain the sampled graph, define snapshot `node_filter`/`edge_filter` as appropriate, not just a seed list. The supported logical and array-native workflows are described separately in the library's graph adapter and sampling guides.

6.3 Theory and further reading

- SPARQL uses basic graph pattern matching by default. The separate SPARQL 1.1 Entailment Regimes recommendation defines extensions for selected entailment relations. A SPARQL endpoint does not automatically implement OWL inference. GestaltDB exposes Cypher plus explicit reasoning adapters, not a SPARQL endpoint.
- Graph query language compares the traversal idioms (pattern matching, path queries, fanout control) that both Cypher clauses and `expand_edge_types` sampling implement in their own layer.

6.4 Check your understanding

Question. We use inferred people as seeds for a snapshot built from stored edges. Does the sampler's next hop automatically traverse inferred relations? Does fanout 10 necessarily return ten distinct neighbors?

Answer. No. A seed handoff does not change snapshot topology. Fanout is a cap; member quotas, overlap, and sparse populations can return fewer endpoints. Neither a proof count nor a parallel-edge count is implicitly a sampling weight.

CHAPTER

SEVEN

THE COST OF REASONING: WHAT BECOMES EXPENSIVE, AND WHY

7.1 Why a fast rule is not necessarily a fast application

Our worked example adds two facts almost immediately. It is easy to conclude that scaling merely requires a faster rule loop. In an actual application, however, the input must be read, decoded, authenticated, mapped, and sometimes converted into another representation before the loop starts. An explanation may then need to be built, and publication may require another consistency check. Optimizing the middle cannot remove the surrounding costs.

A useful way to reason about the whole pipeline is:

$$T_{\text{total}} = T_{\text{extract}} + T_{\text{compile}} + T_{\text{convert}} + T_{\text{fixpoint}} + T_{\text{evidence}} + T_{\text{publish}}$$

These are conceptual terms, not a promise that every executor exposes a separate timer for each. The equation helps ask the right question. If authentication dominates a tiny query, parallelizing a join will barely change response time. If many queries reuse one ontology, amortizing compilation may matter more than a small per-round improvement. Measure the workload you actually repeat.

7.2 Joins: avoid combinations before making them faster

The work-location rule joins appointments to locations on organization ID. A naive nested loop compares every appointment with every location. If the organization is already bound to Acme, a lookup into a matching index bucket can eliminate most comparisons. This is why binding propagation and index selection are foundational reasoning capabilities, not cosmetic optimizations.

The default indexed Python evaluator exploits such bindings. Prepared plans can reuse scheduling and inspection work; a columnar executor can use native join kernels over compact columns. Neither approach wins on every workload. Converting a handful of facts into a heavy runtime may cost more than the joins it saves. Highly skewed keys can create large intermediate relations even when the final answer is small. Reordering a join changes work, not the mathematical set of answers, and explanations must preserve the meaning of premise positions.

Compact batches also have a boundary. A binary row of two uint64 IDs uses 16 payload bytes, but the dictionary, reverse maps, temporary Python objects, join

state, and support indexes still occupy memory. Arrow imports copy and canonicalize; little-endian exports can share immutable column payloads, while wrappers allocate and downstream consumers may copy. “Columnar” and “zero-copy” are not synonyms. See the library’s batch reference for ownership details.

7.3 Some costs are inherent in the requested answer

Consider a chain of `n` organizations linked by `partOf`. Computing every reachable pair yields `n(n-1)/2` distinct pairs. The input is linear in `n`; the full closure is quadratic. No improvement in index lookup can store that explicit output in constant memory. A selective query, a compressed structure, or a different interface may avoid producing it, but that is a change in the requested artifact, not a faster way to materialize the same rows for free.

Proofs and updates add further output. An answer may have several supports; keeping enough of them for sound deletion is more work than retaining one witness. Temporal results may split one true interval into many evidence segments. A database with few entities can consequently still have expensive maintenance or explanation workloads.

7.4 Full recomputation versus maintenance

If one isolated appointment changes, rebuilding everything seems wasteful. Dependency indexes can identify potentially affected consequences, and a support-aware algorithm can preserve unaffected answers. This is the motivation for incremental maintenance. It is not a universal shortcut: a change near the root of a dense hierarchy can affect most of the closure, and tracking supports has its own cost. The crossover between local maintenance and rebuilding should be measured rather than guessed.

Correctness remains non-negotiable. Counts alone can preserve a self-supporting cycle incorrectly after the original assertion disappears. A supported deletion algorithm must remove such unsupported consequences or explicitly fall back to rematerialization. A faster partial or stale result must not be labeled a complete current closure.

7.5 Reading measurements without overgeneralizing

GestaltDB’s documented comparison of indexed Python and its experimental Polars closure path illustrates the trade-off. In one historical Python 3.13 comparison with three fresh workers per cell, dense N=128 pipeline medians were 534.73ms and 46.44ms respectively. Sparse recursive N=1024 also favored Polars, but hierarchy, duplicate-evidence, and small N=16 cases favored Python. Peak worker RSS was approximately 54 MiB for Python versus 127 MiB for Polars. These are workload observations, not a promise about your machine or about proof/maintenance modes that the prototype does not provide.

The timings include compilation, conversion, fixpoint, and decoding but exclude package imports and fixture construction. That definition matters as much as the

numbers. The complete tables, scope, and reproduction commands are in the library's Polars guide; there is no reason to memorize the figures here.

When evaluating your own pipeline, distinguish cold-start latency from reused execution, final payload from peak process RSS, and closure-only results from evidence-bearing ones. Cooperative cancellation stops at supported checkpoints; it does not interrupt an already-running native kernel or provide a hard process-memory guarantee. Such requirements need an appropriate execution boundary, not merely another row counter.

7.6 Check your understanding

Question. A native join kernel is ten times faster in isolation. Must the application also become ten times faster? Can a chain's complete reachability closure use linear explicit row storage?

Answer. No to both. Extraction, conversion, evidence, and publication remain, so the end-to-end improvement depends on their share of the cost. The chain's quadratic number of distinct output pairs requires corresponding explicit row space; avoiding those rows means choosing another answer representation or a more selective question.

CHAPTER

EIGHT

CHOOSING A SUPPORTED REASONING WORKFLOW

8.1 Start with the question, not with a standards label

We began with a small question about Alice and built the pieces needed to answer it: a graph, an explicit meaning for its vocabulary, an engine for implications, a consistent source view, and a way to consume the result. It is tempting to call the combined system "an OWL reasoner" and assume every later semantic task is covered. That would erase the most important design boundary: different questions require different languages and guarantees.

If the application needs class inheritance, domains/ranges, inverses, transitivity, or selected binary property chains over named entities, the ontology compiler and positive binary evaluator are a natural starting point. They provide a deliberately bounded fragment, not complete RDFS or OWL. A standard's name in a diagram is not a conformance certificate.

If the application needs typed literal comparisons, n-ary relations, or supported aggregates, inspect the separate generic typed API. If it needs absence-based rules, determine which predicates and contexts can legitimately be treated as closed, then check the scoped stratified-negation contract. Neither feature appears automatically in the binary ontology engine merely because another module supports it.

8.2 Some problems need a different engine family

Suppose the question changes from "What follows from these appointments?" to "Which of several possible appointment assignments satisfy all our constraints?" That is a search over possible solutions, not simply a positive closure. An answer-set solver such as Clingo addresses such choices under its own semantics; it is not a drop-in replacement with identical answers and resource behavior.

Likewise, arbitrary anonymous class restrictions, unrestricted existential generation, cardinalities, and complete OWL DL satisfiability do not follow from the selected named-entity fragment. GestaltDB has separately gated experimental capabilities, including bounded EL work, but their existence does not expand the compiler used in the introductory examples. Start from the required constructs and inspect their exact supported semantics. See the library's profile and EL guides for those boundaries.

Validation is another distinct need. "Every record must contain a field" is not the same statement as an ontology domain implication. Selected conflict reports are

useful, but should not be sold as a complete shapes validator or a general proof that an ontology is satisfiable. Similarly, deterministic proofs do not constitute a probability model. Pick an explicit validation or confidence contract rather than inferring one from a familiar class name.

8.3 An available feature may not compose with every other one

Combining independently useful features is often the hardest engineering work. A snapshot-local equality mapping, for example, must not be reused after the snapshot changes as though identities had been physically merged forever. Temporal equality may need different representatives in different segments. A literal value must not silently become an entity ID.

The positive binary maintenance API similarly does not cover every generic rule feature. Changed programs, closed scopes, negation, aggregates, and equality splits require the documented explicit recomputation path rather than an unsupported incremental update. Closure checkpoints and bounded one-step ranked witnesses are not the same as durable resumable database-delta sessions or globally minimal recursive proof trees.

The same caution applies to execution. Compact batches are not a native reasoner; a worker/spill policy object is not an out-of-core runtime; a closure-only Polars result is not a proof-bearing publication generation. For each workflow, check the whole chain from input identity through semantics, limits, evidence, and consumption. Unsupported requested guarantees should produce an explicit failure, not a plausible-looking weaker result.

8.4 Integration pain points to plan for

The tutorial's explicit boundaries make behavior understandable, but they also leave application work. Choosing which extracted facts are relevant, binding a selective goal to a pinned database read, persisting a long-running incremental session, and exposing authenticated inferred results to query/sampling consumers are separate integration problems. Do not budget for them as if an in-memory `evaluate` call supplied an entire managed service.

Performance evidence has boundaries too. Native decoded evidence helpers exist, but small fixtures cannot establish a large-scale throughput advantage. A new optimization may improve lookup work while leaving full-state authentication, output size, or alternative-support storage dominant. Chapter 7 explains why the end-to-end measurement matters.

These limitations do not make reasoning useless. They tell us where it is useful: explicit, testable consequences over a declared fragment, with visible boundaries around storage, evidence, time, and consumption. Start with that complete small workflow; add richer semantics or acceleration only when a concrete question requires it.

8.5 What you should now be able to explain

Return to the directory question. Alice's appointment is stored data; the Employee-to-Person relationship is a modeling axiom; the rule engine computes membership, with Researcher requiring an extra job-description premise; a pinned adapter binds the input to a source state; and a logical query reads the consequence without silently mutating the stored labels. Changing one of those steps changes either the question, the assumptions, or the execution guarantee. Being able to distinguish them is more valuable than remembering a particular import path.

8.6 Check your understanding

Question. A proposed feature needs typed aggregates, temporal equality, incremental retractions, and proof-bearing publication. Is it enough to find one module advertising each feature?

Answer. No. Check whether the combination is supported by the same input, evaluation, maintenance, and publication contracts. If not, choose a documented complete recomputation strategy, split the workflow explicitly, or use a different engine whose semantics and guarantees meet the requirement. Do not compose feature names into a guarantee the system never made.

CHAPTER

NINE

GLOSSARY: THE DISTINCTIONS THAT MATTER

Entity
: A thing with identity in the modeled domain: Alice, Acme Lab, or a paper. A node ID is an application's identifier for it, not a complete definition of what that entity means.

Relationship / predicate
: A named relation such as `worksFor`. A stored property-graph edge has its own identity and properties; a binary logical fact is a subject/predicate/ object tuple. Several physical edges may project to one set-valued fact.

Class / instance
: `Researcher` is a class; Alice is an instance. "Alice is a Researcher" is membership, while "Researcher is a subclass of Employee" relates two classes. Those are different statements, not interchangeable edges.

Label / property
: A native node label supports grouping and lookup in the property graph. A property stores a value such as a display name. Neither gets OWL semantics merely by naming it after an ontology term; an explicit mapping is needed.

Graph database
: A database interface and storage/query system for graph-shaped data. Storage, indexing, transactions, and query execution are database concerns.

Knowledge graph
: A graph-based representation of entities and their relationships, intended to convey domain knowledge with identifiable terms and useful meaning. There is no single universally enforced definition. It may use property graphs or RDF, and need not have an OWL reasoner or a formal ontology.

RDF / IRI
: RDF represents a graph as a set of triples. IRIs identify terms; blank nodes and literals are also available in permitted positions. An IRI is not an instruction to fetch a URL. A local Python string is not automatically an IRI.

Ontology / axiom
: An ontology supplies explicit vocabulary and semantic statements. An axiom is one such statement, for example `SubClassOf("Employee", "Person")`. This tutorial uses a named, deliberately limited fragment, not all of OWL.

Employment / job description / role refinement
: `worksFor` denotes an employment relationship here, not mere presence at a workplace. `hasJobDescription` supplies separate job evidence. Employee

membership alone does not imply Researcher; that finer class requires the classification rule's positive job-description premise. The explicit category `not_researcher` is not a synonym for missing research evidence.

Assertion / consequence / closure
An assertion is supplied as input. A consequence follows under the chosen rules. The closure is the asserted set plus all consequences in the declared fragment. Being in a logical closure does not imply being written to storage.

Query / inference / validation
A query selects or computes an answer from a chosen view. Inference derives consequences under rules. Validation checks selected consistency or data requirements. A language or system can offer more than one of these services; domain/range inference is not a missing-field validation constraint.

Rule body / head / substitution
In `B1 ∧ B2 ⇒ H`, the body contains the premises and the head is the conclusion. A substitution consistently assigns constants to variables. Repeated variables must receive the same value throughout the rule.

Entailment / soundness / completeness
$F \cup P \models q$ means that q follows from the input facts and rules under the declared semantics. Soundness excludes unjustified returned answers; completeness requires all answers of the supported fragment. Neither establishes that the input assumptions are true in the real world.

Fixed point / delta / intermediate representation
A fixed point adds no new facts when the rules run again. A delta is the latest addition used to avoid repeating old join work. An intermediate representation (IR) expresses a task at a boundary—such as rules, plans, or compact relations—without forcing every stage into one data structure.

Open world / explicit negation / closed world
Under an open-world reading, missing evidence is not evidence of falsehood. Explicit negative evidence is a separate assertion. A closed-world scope permits absence-based negation only within its declared boundaries.

Witness / support / provenance
A witness is a selected explanation of a conclusion. A support records a particular rule/premise derivation; alternative supports matter on deletion. Provenance records where assertions or derivations came from and which context they belong to. One witness is not a complete maintenance index.

Snapshot / horizon / publication
A snapshot pins what a read observes; a horizon identifies the selected system-time prefix. Publication is an explicit write of computed results. An array-native sampling snapshot is a different artifact from a database read snapshot or a logical closure.

CHAPTER

TEN

TUTORIAL NOTEBOOKS

Use the environment described in *From Graph Databases to Knowledge Graphs*. Download or open each lab:

- `nb01_subclass_closure.ipynb` — compile `SubClassOf` + `Domain`, infer Employee/Person membership, then add a job-description rule that separately refines employees into researchers; inspect `result.explain`.
- `nb02_property_chains.ipynb` — binary `PropertyChain` + transitivity, origin table back to axiom IDs.
- `nb03_graph_projection.ipynb` — project a pinned LMDB property graph to facts, infer membership, and visualize a caller-owned logical graph without modifying stored labels.

Work through the labs in order if you want hands-on reinforcement. The first isolates the meaning of two axioms; the second shows why combining rules is more interesting than applying each once; the third connects the same semantics to stored data and verifies that inference has not silently changed its labels.

Each lab includes assertions for the expected answers. Read the accompanying explanation before executing the cell, predict the result, then vary a premise or axiom. If an answer changes, identify the rule body that stopped matching. That habit is more useful than rerunning a notebook until it reports success.

Outputs are omitted so that you can produce them in your own environment. The HTML documentation offers downloads; the PDF describes the labs but does not execute or embed a live notebook session.